



FITNESS CENTER

BI System - ITI Graduation Project



OUR TEAM



Mohamed Ghazal



Maryam Salah



Ahmed Khaled



Sherif Khaled



Mohammad Anwar



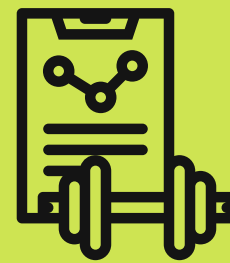
Youssef Radwan



TABLE OF CONTENT



- Project Overview
- Project Workflow
- Tools & Technologies
- ERD & Mapping



- Database Tables
- Database Creation
- Database T-SQL
- RFM Analysis



- SSRS stored Procedures
- SSRS Reports
- Power BI Dashboards
- Website UI & Function



PROJECT OVERVIEW

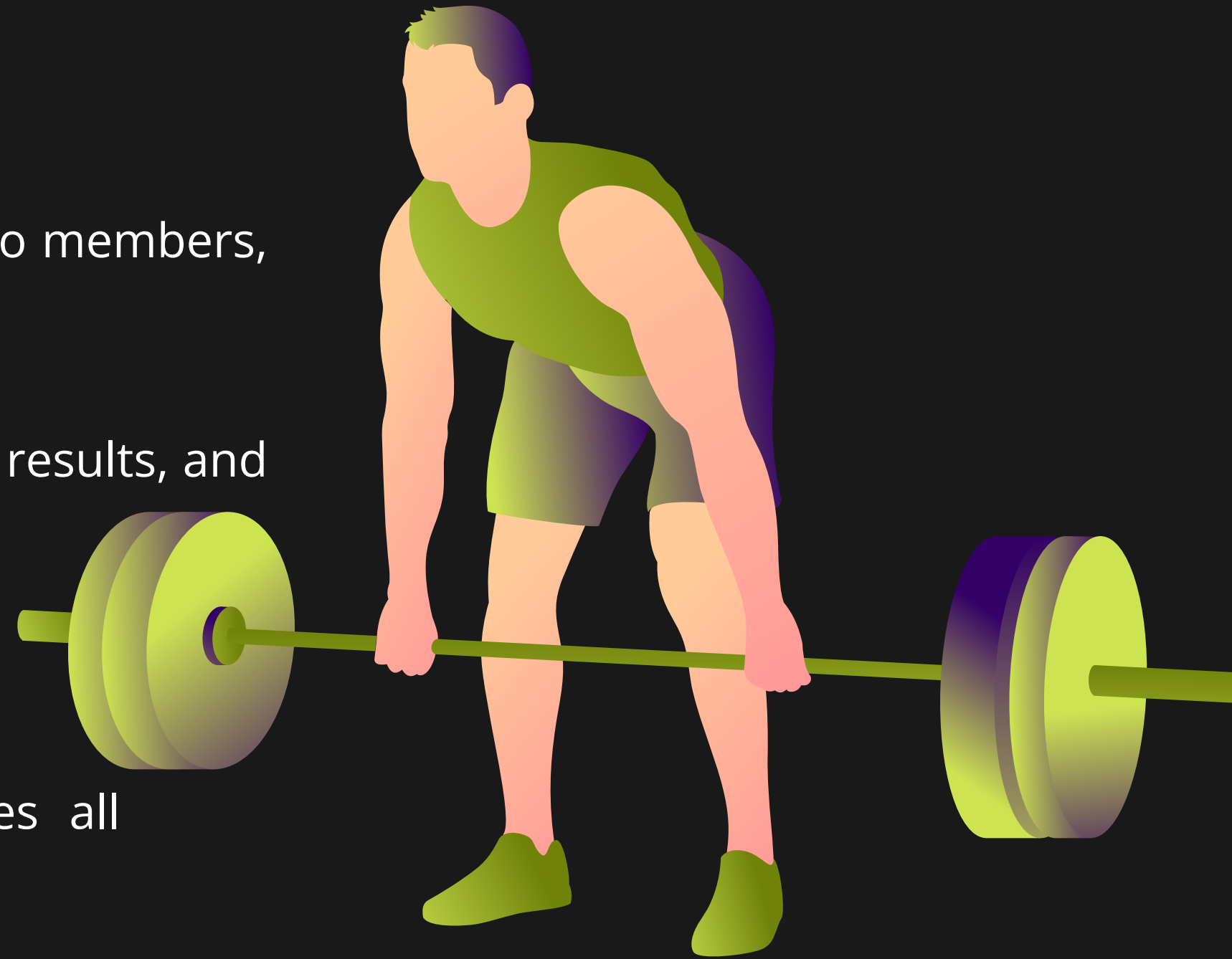
Problem:

Fitness centers deal with large amounts of scattered data related to members, attendance, branches, and performance.

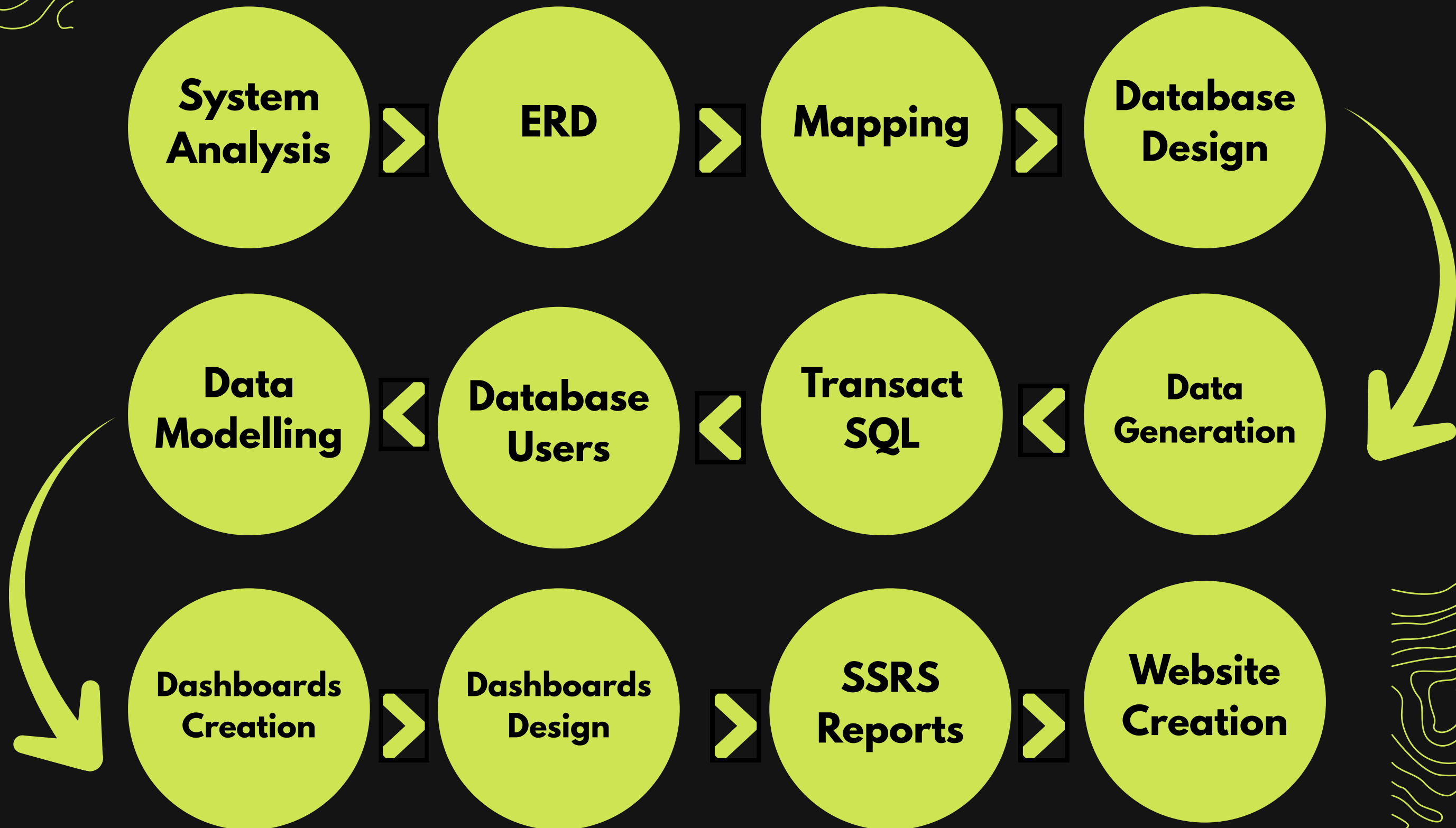
This lack of organization makes it difficult to track trends, measure results, and make informed decisions.

Solution:

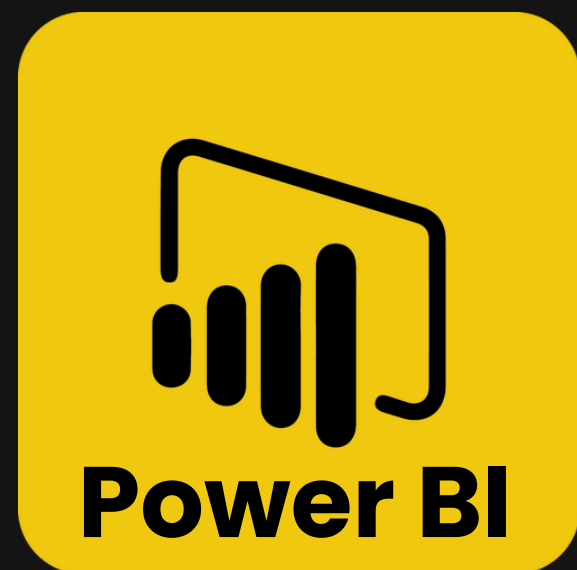
The project provides an end to end BI solution that organizes all operational data in one structured place.



PROJECT WORKFLOW



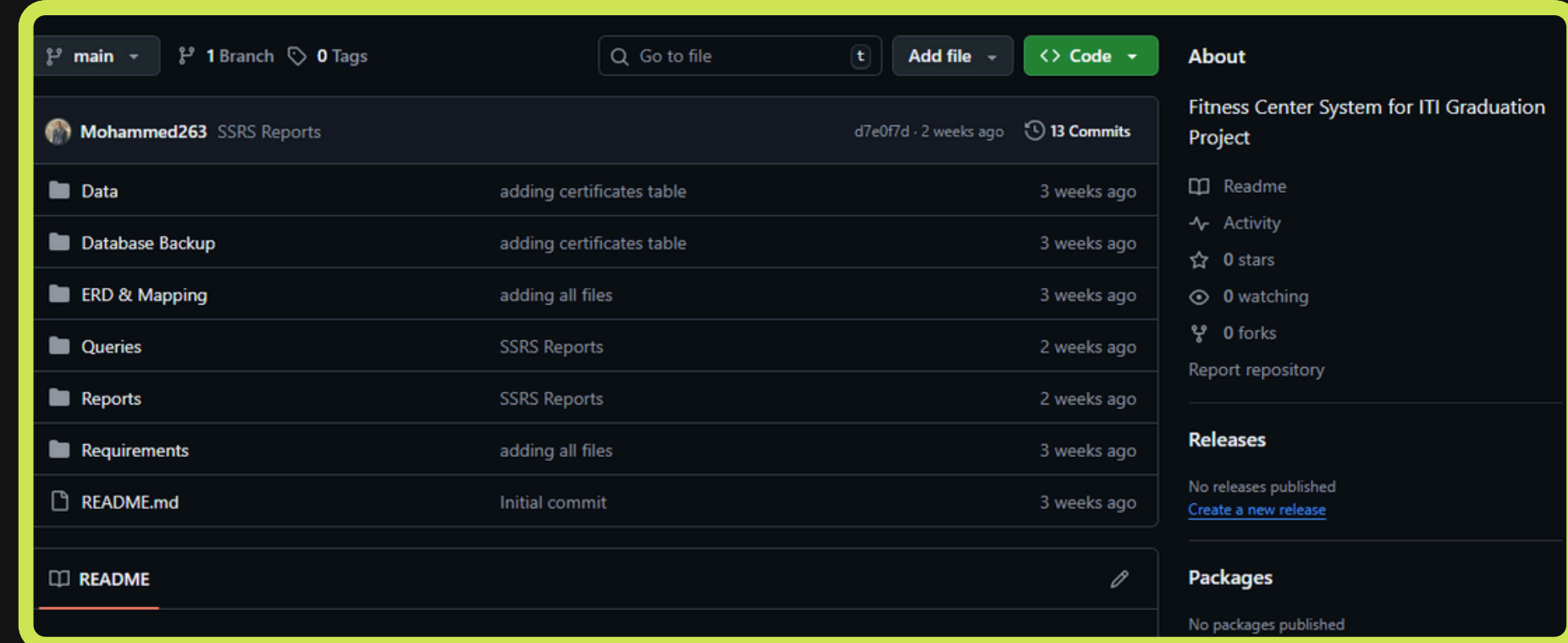
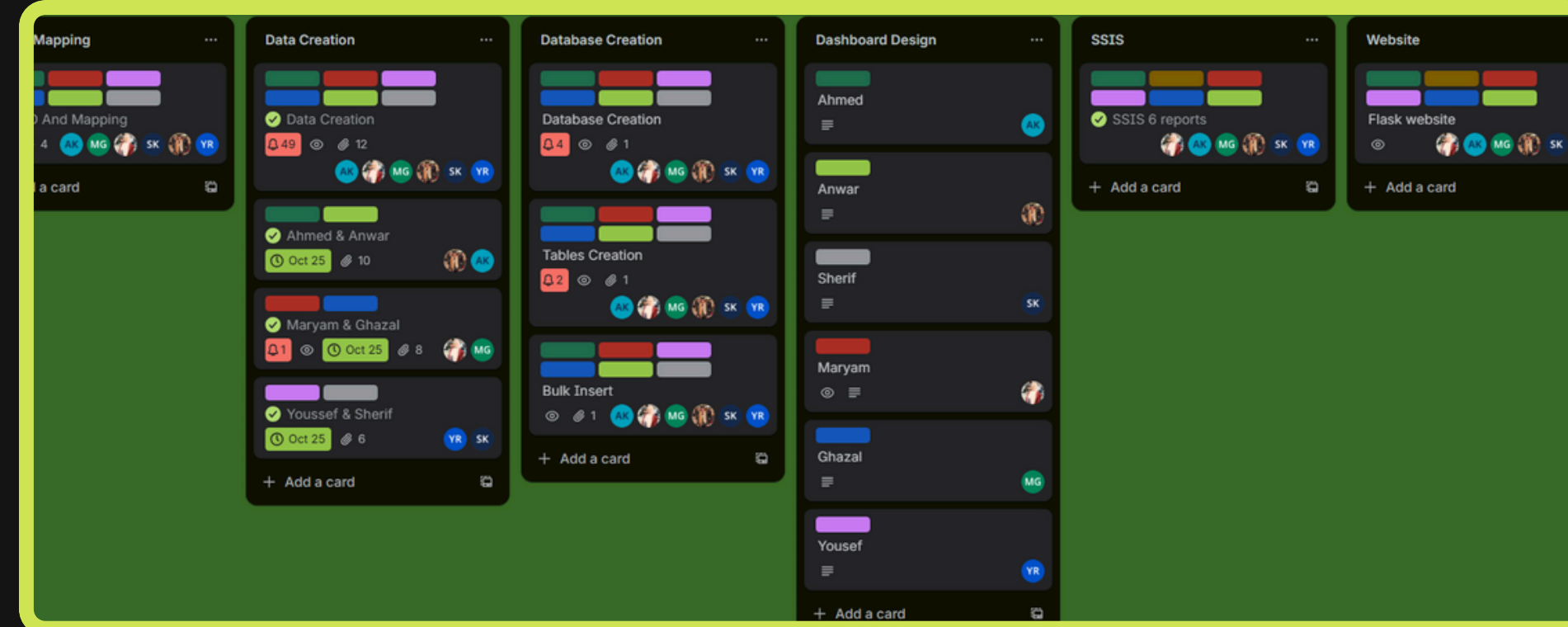
TOOLS & TECHNOLOGIES



Collaboration & Source Control

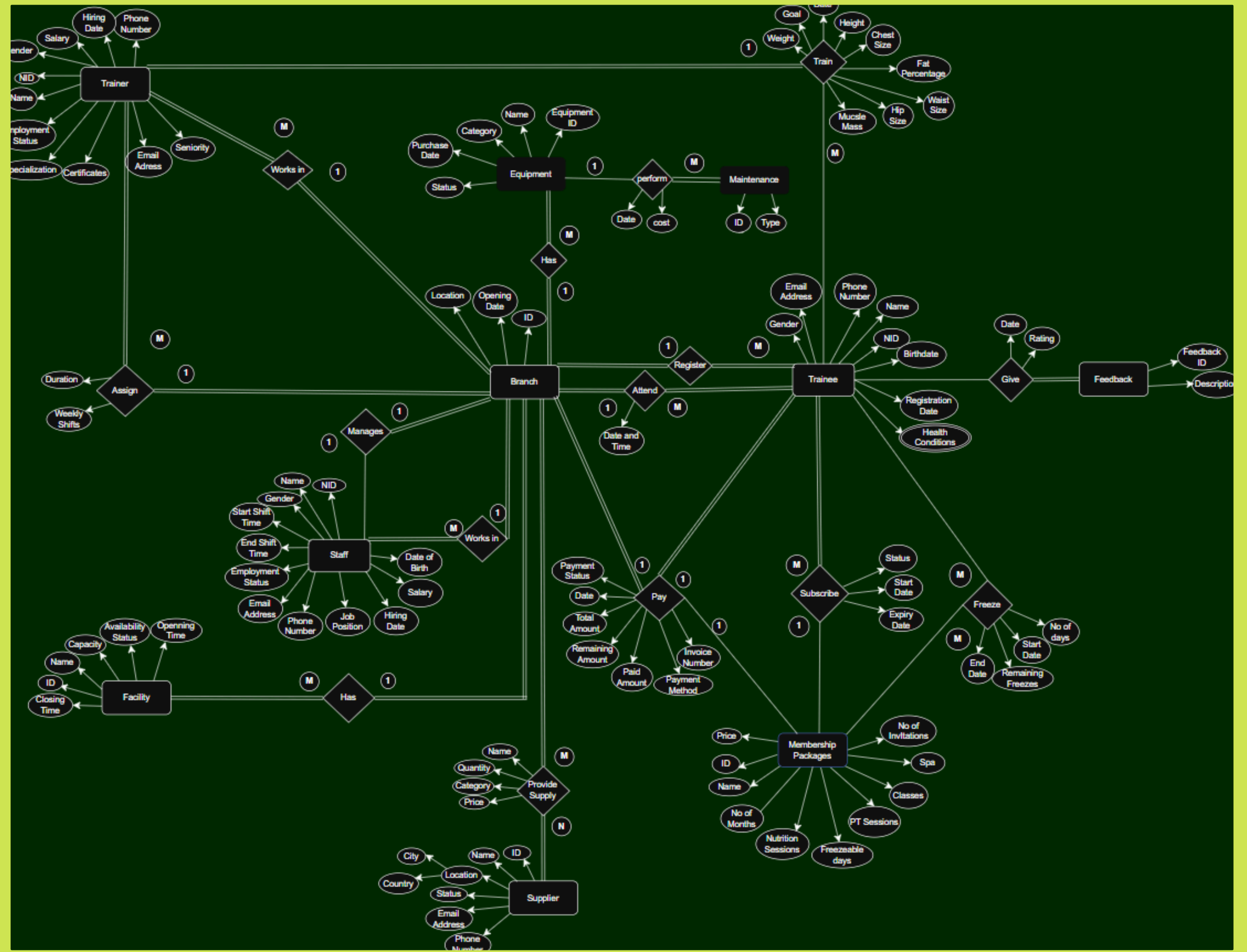
Trello

GitHub



Entities: 10

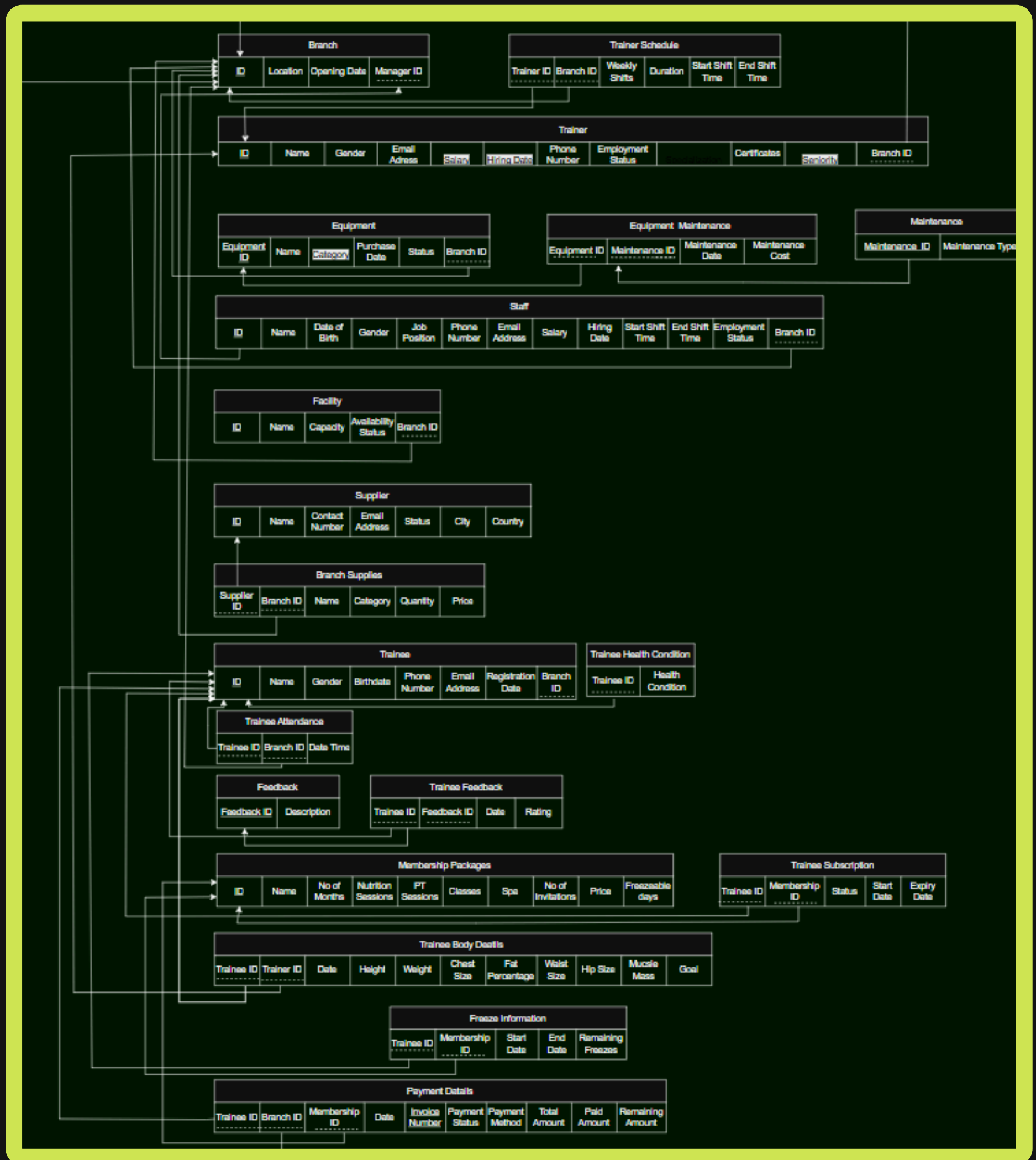
RelationShips: 15



MAPPING

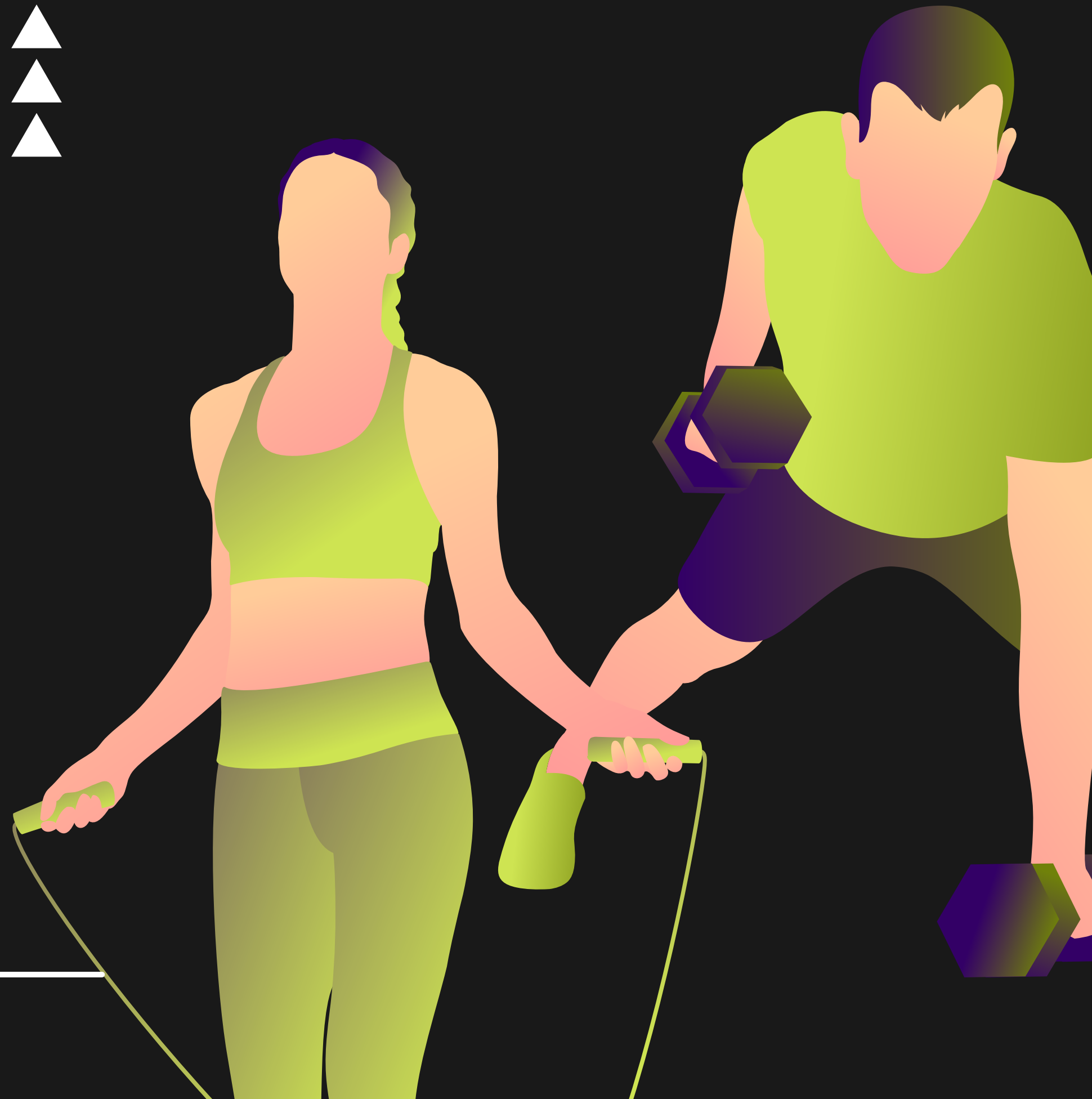
Tables: 20

Satisfies the Third Normal Form



DATABASE TABLES

- Branch
- Equipment
- Equipment Maintenance
- Facility
- Feedback
- Freeze
- Maintenance
- Membership Packages
- Payment Details
- Staff
- Supplier
- Supplies
- Trainee
- Trainee Attendance
- Trainee Body Details
- Trainee Feedback
- Trainee Health conditions
- Trainee Subscription
- Trainer
- Trainer Certificates
- Trainer Schedule





DATABASE CREATION

we created our database in three phases

01

Creating Constraints, Rules and Indexes to assure validation

02

Generating data using AI, Testing the data, Then inserting the data

03

Program the database via stored procedures, Triggers and defaults



Constraints & Rules & Defaults

CONSTRAINTS

```
-- create unique constraint on EmailAddress
Alter Table Trainer Add Constraint UQ_TrainerEmail UNIQUE (EmailAddress)
Alter Table Staff Add Constraint UQ_StaffEmail UNIQUE (EmailAddress)
Alter Table Trainee Add Constraint UQ_TraineeEmail UNIQUE (EmailAddress)
Alter Table Supplier Add Constraint UQ_SupplierEmail UNIQUE (EmailAddress)

-- create unique constraint on PhoneNumber
Alter Table Trainer Add Constraint UQ_TrainerPhone UNIQUE (PhoneNumber)
Alter Table Staff Add Constraint UQ_StaffPhone UNIQUE (PhoneNumber)
Alter Table Trainee Add Constraint UQ_TraineePhone UNIQUE (PhoneNumber)
Alter Table Supplier Add Constraint UQ_SupplierPhone UNIQUE (ContactNumber)

-- create unique constraint on Name
Alter Table Supplier Add Constraint UQ_SuppName UNIQUE (Name)
Alter Table MembershipPackages Add Constraint UQ_MembName UNIQUE (Name)
Alter Table Maintenance Add Constraint UQ_MainName UNIQUE (Type)
Alter Table Feedback Add Constraint UQ_FedName UNIQUE (Description)

-- create unique constraint on InvoiceNumber
Alter Table PaymentDetails Add Constraint UQ_Invoice UNIQUE (InvoiceNumber)

-- create check constraint on feedback rating
Alter Table TraineeFeedback Add Constraint C_Rating CHECK ((Rating) between 1 and 5)

-- create check constraint on Payment Method
Alter Table PaymentDetails Add Constraint C_PayMethod
CHECK (PaymentMethod in ('Credit Card','Cash','Instapay'))

-- create check constraint on Date
Alter Table TraineeSubscription Add Constraint C_SupDate CHECK (ExpiryDate > StartDate)
Alter Table Freeze Add Constraint C_FrezDate CHECK (EndDate > StartDate)
```

RULES

```
Create Rule Gen_Rule As @Gen_Col in ('M','F')

Sp_binrule Gen_Rule , 'Trainer.Gender'
Sp_binrule Gen_Rule , 'Staff.Gender'
Sp_binrule Gen_Rule , 'Trainee.Gender'
```

DEFAULTS

```
Create Default PersonStatus As 'Active'
sp_bindefault PersonStatus , 'TraineeSubscription.Status'
sp_bindefault PersonStatus , 'Trainer.EmploymentStatus'
sp_bindefault PersonStatus , 'Staff.EmploymentStatus'
sp_bindefault PersonStatus , 'Supplier.Status'
sp_bindefault PersonStatus , 'TraineeSubscription.Status'
```

```
Create Default EquipmentStatus As 'Operational'
sp_bindefault EquipmentStatus , 'Equipment.Status'
```

```
Create Default FacilityStatus As 'Operational'
sp_bindefault FacilityStatus , 'Equipment.Status'
```

```
Create Default TimeNow As GETDATE()
sp_bindefault TimeNow , 'TraineeAttendance.DateTime'
```

```
Create Default DateNow As CAST(GETDATE() AS DATE)
sp_bindefault DateNow , 'TraineeFeedback.Date'
sp_bindefault DateNow , 'TraineeBodyDetails.Date'
sp_bindefault DateNow , 'PaymentDetails.Date'
```

Generating & Inserting Data

BULK INSERT

- Generated Data via AI (**Chat GPT, Gemini**) by adjusting the appropriate prompts to generate tables
- Ensured Data validation using **Excel** to check the cleanness of the data and data types Compatibility
- Inserted data into the database using **Bulk Insert** in SQL Server

Google
Gemini



```
--insert Data into Branch Table
BULK INSERT Branch
FROM 'D:\ITI\Graduation Project\Data\Branch.csv'
WITH (
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '0x0a',
    FIRSTROW = 2
)
select * from Branch
delete from Branch
```



```
-- create index on trainee phone number  
create nonclustered index  
ix_traineephone  
on trainee (PhoneNumber)
```



INDEXES

Created a non Clustered Index to on the trainee phone number to to improve and speed up the search process based on that field





STORED PROCEDURES

Created stored procedures to handle all **CRUD** operations (Create, Read, Update, Delete).

This helps reduce repetitive code, improve performance, and ensure data is managed securely and consistently.

```
--trainer (Add-update-schedule-certificates)
--staff    ((Add-update)
--Membership (Add-update-delete)
--supplier  ((Add-update)
--supplies  ((Add-update)
--Equipment ((Add-update,equipmentmaintenance)
--Maintenance (Add-update-delete)
--Branch    (Add-update-delete)
--Facility   (Add-update-delete)
```



Stored Procedures for Adding

ADD BRANCH

```
CREATE OR ALTER PROCEDURE AddBranch
    @ID INT,
    @Location VARCHAR(20),
    @OpeningDate DATE,
    @ManagerID INT
AS
BEGIN
    SET NOCOUNT ON;

    -- Check if Branch already exists
    IF EXISTS (SELECT 1 FROM Branch WHERE ID = @ID)
    BEGIN
        RAISERROR('Branch with this ID already exists.', 16, 1);
        RETURN;
    END

    INSERT INTO Branch (ID, [Location], OpeningDate, ManagerID)
    VALUES (@ID, @Location, @OpeningDate, @ManagerID);

    SELECT 'Branch added successfully.' AS Message;
END;
```

ADD SUPPLIER

```
CREATE OR ALTER PROC add_supplier
    @ID INT,
    @Name VARCHAR(100),
    @ContactNumber VARCHAR(20),
    @EmailAddress VARCHAR(100),
    @Status VARCHAR(50),
    @City VARCHAR(100),
    @Country VARCHAR(100)
AS
BEGIN
    IF EXISTS (SELECT 1 FROM Supplier WHERE ID = @ID)
    BEGIN
        THROW 50041, 'Supplier ID already exists.', 1;
    END

    INSERT INTO Supplier
    (ID, Name, ContactNumber, EmailAddress, Status, City, Country)
    VALUES
    (@ID, @Name, @ContactNumber, @EmailAddress, @Status, @City, @Country)
END;
```

Stored Procedures for Updating

UPDATE TRAINER

```
CREATE OR ALTER PROC update_trainer
    @ID INT,
    @Name VARCHAR(50) = NULL,
    @Gender VARCHAR(1) = NULL,
    @EmailAddress VARCHAR(50) = NULL,
    @PhoneNumber VARCHAR(20) = NULL,
    @EmploymentStatus VARCHAR(20) = NULL,
    @Specialization VARCHAR(100) = NULL,
    @Salary FLOAT = NULL,
    @HiringDate DATE = NULL
AS
BEGIN
    IF NOT EXISTS (SELECT 1 FROM Trainer WHERE ID = @ID)
    BEGIN
        THROW 50010, 'Trainer does not exist.', 1
    END

    UPDATE Trainer
    SET
        Name = ISNULL(@Name, Name),
        Gender = ISNULL(@Gender, Gender),
        EmailAddress = ISNULL(@EmailAddress, EmailAddress),
        PhoneNumber = ISNULL(@PhoneNumber, PhoneNumber),
        EmploymentStatus = ISNULL(@EmploymentStatus, EmploymentStatus),
        Specialization = ISNULL(@Specialization, Specialization),
        Salary = ISNULL(@Salary, Salary),
        HiringDate = ISNULL(@HiringDate, HiringDate)
    WHERE ID = @ID
END
```

UPDATE TRAINER SCHEDULE

```
CREATE OR ALTER PROC update_trainer_schedule
    @TrainerID INT,
    @BranchID INT = NULL,
    @WeeklyShifts INT = NULL,
    @Duration INT = NULL,
    @StartShiftTime TIME = NULL,
    @EndShiftTime TIME = NULL
AS
BEGIN
    IF NOT EXISTS (SELECT 1 FROM TrainerSchedule WHERE TrainerID = @TrainerID)
    BEGIN
        THROW 50011, 'Trainer schedule does not exist.', 1
    END

    UPDATE TrainerSchedule
    SET
        BranchID = ISNULL(@BranchID, BranchID),
        WeeklyShifts = ISNULL(@WeeklyShifts, WeeklyShifts),
        Duration = ISNULL(@Duration, Duration),
        StartShiftTime = ISNULL(@StartShiftTime, StartShiftTime),
        EndShiftTime = ISNULL(@EndShiftTime, EndShiftTime)
    WHERE TrainerID = @TrainerID;
END
```


Stored Procedures for Deleting

DELETE PACKAGE

```
CREATE OR ALTER PROC delete_membership_package
    @ID INT
AS
BEGIN

    IF NOT EXISTS (SELECT 1 FROM MembershipPackages WHERE ID = @ID)
    BEGIN
        THROW 50032, 'Membership Package ID does not exist.', 1
    END

    -- Delete
    DELETE FROM MembershipPackages
    WHERE ID = @ID
END
```

DELETE FEEDBACK

```
CREATE OR ALTER PROCEDURE DeleteFeedback
    @ID INT
AS
BEGIN
    SET NOCOUNT ON;

    IF NOT EXISTS (SELECT 1 FROM Feedback WHERE ID = @ID)
    BEGIN
        RAISERROR('Feedback not found.', 16, 1);
        RETURN;
    END

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Delete dependent records first (important)
        DELETE FROM TraineeFeedback WHERE FeedbackID = @ID;

        DELETE FROM Feedback WHERE ID = @ID;

        COMMIT TRANSACTION;

        SELECT 'Feedback deleted successfully.' AS Message;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        RAISERROR('Cannot delete Feedback due to related records.', 16, 1);
    END CATCH
END;
```

Triggers

1. PreventAttendance_AfterExpiry

Table: TraineeAttendance

Fires: AFTER INSERT

Purpose:

Prevents a trainee from logging attendance after their subscription has expired.

If someone tries to attend on a date later than their subscription expiry, the trigger rolls back the insert and shows an error.

```
-- Trigger to prevent the subscriber from attending after the subscription ends
Create or Alter Trigger PreventAttendance_AfterExpiry
on TraineeAttendance
After Insert
AS
Begin
    if Exists (
        select 1 from inserted inner join TraineeSubscription
        on TraineeSubscription.TraineeID=inserted.TraineeID
        and TraineeSubscription.StartDate=
        (select max(TraineeSubscription.StartDate) from TraineeSubscription
        where TraineeSubscription.TraineeID=inserted.TraineeID)
        where CAST(getdate() as Date) > TraineeSubscription.ExpiryDate
    )
    Begin
        rollback
        RAISERROR('the subscription has expired.', 16, 1);
    End
End
```

Triggers

2. HandleFreeze

Table: Freeze

Fires: AFTER INSERT, UPDATE

Purpose:

It handles all freeze logic for subscriptions:

It performs 3 main functions:

1. Validates first freeze: Prevents a trainee from taking more freeze days than allowed on the first freeze in a subscription.

2. Validates remaining freezes: If the trainee already used some freeze days, the trigger ensures the newly inserted freeze does not exceed the remaining allowed freeze days.

3. Updates freeze and subscription data automatically: Calculates and sets the freeze EndDate, Calculates RemainingFreezes, Extends the trainee's subscription ExpiryDate based on freeze days.

```
CREATE OR ALTER TRIGGER HandleFreeze
ON Freeze
AFTER INSERT, UPDATE
AS
BEGIN
    -- check if it's the first freeze in the subscription
    IF EXISTS (
        SELECT 1
        FROM inserted
        INNER JOIN TraineeSubscription
            ON TraineeSubscription.TraineeID = inserted.TraineeID
            AND TraineeSubscription.MembershipID = inserted.MembershipID
            AND inserted.StartDate BETWEEN TraineeSubscription.StartDate AND TraineeSubscription.ExpiryDate
        INNER JOIN MembershipPackages
            ON MembershipPackages.ID = inserted.MembershipID
        WHERE NOT EXISTS (
            SELECT 1
            FROM Freeze
            INNER JOIN TraineeSubscription
                ON TraineeSubscription.TraineeID = Freeze.TraineeID
                AND TraineeSubscription.MembershipID = Freeze.MembershipID
                AND Freeze.StartDate BETWEEN TraineeSubscription.StartDate AND TraineeSubscription.ExpiryDate
            WHERE Freeze.TraineeID = inserted.TraineeID
                AND Freeze.MembershipID = inserted.MembershipID
                AND Freeze.StartDate < inserted.StartDate
        )
        AND inserted.TakenFreezes > MembershipPackages.FreezableDays
    )
    BEGIN
        RAISERROR('Taken freeze exceeds allowed FreezableDays for first freeze in this subscription', 16, 1);
        ROLLBACK TRANSACTION;
        RETURN;
    END

    -- check if he has past freeze in same subscription
    IF EXISTS (
        SELECT 1
        FROM inserted
        INNER JOIN TraineeSubscription
            ON TraineeSubscription.TraineeID = inserted.TraineeID
            AND TraineeSubscription.MembershipID = inserted.MembershipID
            AND inserted.StartDate BETWEEN TraineeSubscription.StartDate AND TraineeSubscription.ExpiryDate
        INNER JOIN MembershipPackages
            ON MembershipPackages.ID = inserted.MembershipID
        WHERE EXISTS (
            SELECT 1
            FROM Freeze
            INNER JOIN TraineeSubscription
                ON TraineeSubscription.TraineeID = Freeze.TraineeID
                AND TraineeSubscription.MembershipID = Freeze.MembershipID
                AND Freeze.StartDate BETWEEN TraineeSubscription.StartDate AND TraineeSubscription.ExpiryDate
            WHERE Freeze.TraineeID = inserted.TraineeID
                AND Freeze.MembershipID = inserted.MembershipID
                AND Freeze.StartDate < inserted.StartDate
            AND inserted.TakenFreezes >
                (MembershipPackages.FreezableDays -
                 (SELECT SUM(Freeze.TakenFreezes)
                  FROM Freeze
                  INNER JOIN TraineeSubscription
                      ON TraineeSubscription.TraineeID = Freeze.TraineeID
                      AND TraineeSubscription.MembershipID = Freeze.MembershipID
                      AND Freeze.StartDate BETWEEN TraineeSubscription.StartDate AND TraineeSubscription.ExpiryDate
                  WHERE Freeze.TraineeID = inserted.TraineeID
                      AND Freeze.MembershipID = inserted.MembershipID
                      AND Freeze.StartDate < inserted.StartDate))
        )
    )
    BEGIN
        -- Update logic for remaining freezes and subscription data
    END
END
```

Triggers

3. *TR_PreventBranchDelete*

Table: Branch

Fires: INSTEAD OF DELETE

Purpose:

Prevents deleting a branch if it is still linked to (Equipment, Facilities).

If any related records still exist, the trigger blocks the deletion and displays an error.

```
-- Trigger to Prevent Deleting a Branch That Still Has Related Records
CREATE OR ALTER TRIGGER TR_PreventBranchDelete
ON Branch
INSTEAD OF DELETE
AS
BEGIN
    SET NOCOUNT ON;

    -- Check Equipment
    IF EXISTS (
        SELECT 1
        FROM Equipment E
        INNER JOIN deleted D ON E.BranchID = D.ID
    )
    BEGIN
        RAISERROR('Cannot delete Branch because it still contains Equipment.', 16, 1);
        RETURN;
    END

    -- Check Facilities
    IF EXISTS (
        SELECT 1
        FROM Facility F
        INNER JOIN deleted D ON F.BranchID = D.ID
    )
    BEGIN
        RAISERROR('Cannot delete Branch because it still contains Facilities.', 16, 1);
        RETURN;
    END

    -- If nothing prevents deletion -> Execute
    DELETE FROM Branch
    WHERE ID IN (SELECT ID FROM deleted);
END;
```


Triggers

4.TR_UpdateEquipmentStatus_AfterMaintenance

Table: EquipmentMaintenance

Fires: AFTER INSERT

Purpose:

Automatically updates the status of the equipment to "Under Maintenance" when a maintenance record is inserted.

```
--Auto-Update Equipment Status After Maintenance Insert
CREATE OR ALTER TRIGGER TR_UpdateEquipmentStatus_AfterMaintenance
ON EquipmentMaintenance
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE Equipment
    SET Status = 'Under Maintenance'
    WHERE ID IN (SELECT EquipmentID FROM inserted);
END;
```

Triggers

5. TR_EnforceOneManagerPerBranch

Table: Branch

Fires: FOR INSERT, UPDATE

Purpose:

Ensures that one manager cannot manage more than one branch.

If a manager ID appears in more than one branch, the trigger rolls back the change and raises an error.

```
--Prevent More Than One Branch Having the Same Manager
CREATE OR ALTER TRIGGER TR_EnforceOneManagerPerBranch
ON Branch
FOR INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF EXISTS (
        SELECT ManagerID
        FROM Branch
        WHERE ManagerID IS NOT NULL
        GROUP BY ManagerID
        HAVING COUNT(*) > 1
    )
    BEGIN
        RAISERROR('This manager is already assigned to another branch. Only one branch can have this manager.', 16, 1);
        ROLLBACK TRANSACTION;
        RETURN;
    END
END;
```



RFM ANALYSIS

RFM (Recency, Frequency, Monetary) is a behavioral model used to evaluate member engagement and value.

Recency: How recently a member renewed.

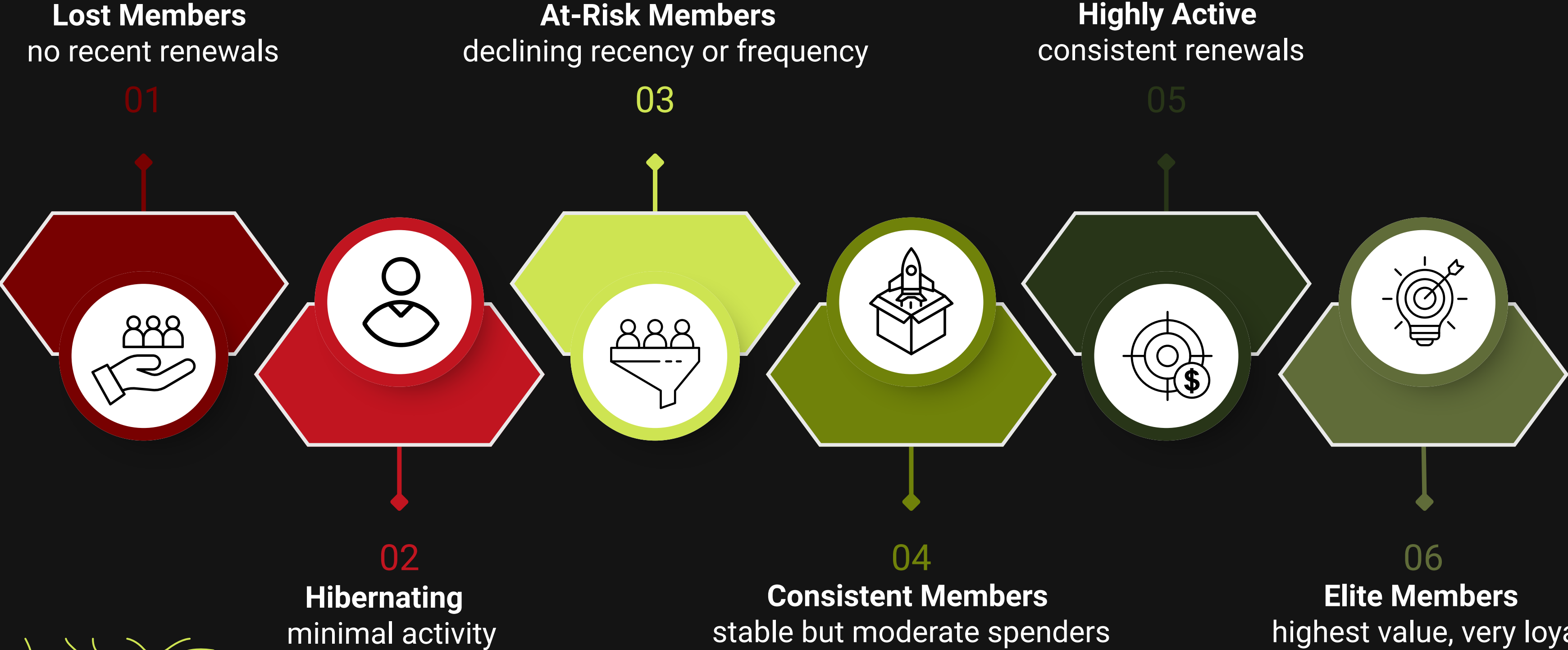
Frequency: How often they renew.

Monetary: How much they spend.

RFM helps identify high-value members, understand loyalty patterns, and detect early signs of inactivity.



RFM Scoring & Segmentation




```

/* RFM Segmentation Query */[...
go
create or alter view rfm_view
as

with rfm_base as(
    select
        TraineeID,
        BranchID,
        max(cast(Date as datetime)) as 'Last_Order_Date',
        count(distinct InvoiceNumber) as 'Invoice_Count',
        sum(CAST(TotalAmount as decimal(18,2))) as 'Total_Price'
    from PaymentDetails
    group by TraineeID,BranchID
),

rfm as(
    select
        TraineeID,
        BranchID,
        DATEDIFF(DAY, Last_Order_Date, GETDATE()) as 'Recency',
        Invoice_Count,
        Total_Price
    from rfm_base
),

-- STEP 1: Get distinct values only
freq_raw as (
    select distinct Invoice_Count
    from rfm
),

-- STEP 2: Rank values from highest to lowest
freq_ranked as (
    select
        Invoice_Count,
        DENSE_RANK() over (order by Invoice_Count asc) as rank_desc
    from freq_raw
),

-- STEP 3: Convert rank → 5 buckets (exactly like NTILE but correct)
frequency_levels as (
    select
        invoice_count,

```

```

        dense_rank() over (order by invoice_count desc) as freq_rank,

        -- final frequency score from 1 to 5
        ceiling(6 - dense_rank() over (order by invoice_count desc)) as frequency_score
    from rfm
    group by invoice_count
),

customer_segemnt_score as (
    select
        rfm.TraineeID,
        rfm.BranchID,
        rfm.Recency,
        rfm.Invoice_Count,
        rfm.Total_Price,
        NTILE(5) over (order by Recency asc) as 'Recency_Score',
        fl.Frequency_Score,
        NTILE(5) over (order by Total_Price desc) as 'Monetary_Score'
    from rfm
    join frequency_levels fl
    on rfm.Invoice_Count = fl.Invoice_Count
)

select css.TraineeID,
css.BranchID,
b.location,
css.Recency_Score,
css.Frequency_Score,
css.Monetary_Score,
(css.Recency_Score + css.Frequency_Score + css.Monetary_Score) as 'Total_Score',
case
    when (css.Recency_Score + css.Frequency_Score + css.Monetary_Score) >= 13 then 'Elite Members'
    when (css.Recency_Score + css.Frequency_Score + css.Monetary_Score) >= 11 then 'Highly Active'
    when (css.Recency_Score + css.Frequency_Score + css.Monetary_Score) >= 8 then 'Consistent Members'
    when (css.Recency_Score + css.Frequency_Score + css.Monetary_Score) >= 6 then 'At-Risk Members'
    when (css.Recency_Score + css.Frequency_Score + css.Monetary_Score) >= 4 then 'Inactive / Hibernating'
    else 'Lost Members'
end as Member_Segment
from customer_segemnt_score css
join Branch b
on css.BranchID = b.ID
--order by Total_Score desc

select * from rfm_view
order by Total_Score desc

```



SSRS REPORTS

Developed a collection of SSRS reports powered by custom stored procedures to support operational, financial, and managerial decision-making across different branches of the fitness center.

Each stored procedure extracts and structures branch-specific data, allowing the reports to be fully dynamic and filterable by one or multiple branches.

These reports provide visibility into **staff performance**, **financial trends**, **operational activities**, and **customer feedback**, helping management monitor the business and make informed decisions.



SSRS Stored Procedures

1. *Trainer Salaries Report*

Shows trainer payroll information, including salary, seniority, and the branch location.

It helps management track trainer distribution and compensation across selected branches.

```
-- Trainer_Salaries

create or alter proc Trainer_Salaries @Branch nvarchar(max)
as
begin
select t.name, t.salary, t.seniority, b.location
from trainer t
join TrainerSchedule ts
on t.id = ts.TrainerID
join branch b
on ts.BranchID = b.id
where b.ID in (SELECT CAST(value AS INT)
               FROM STRING_SPLIT(@Branch, ','))
end
```

SSRS Stored Procedures

2. Staff Salaries Report

Displays details about staff members such as name, salary, job position, employment status, and the branch they belong to.

Used to monitor HR structure and payroll expenses per branch.

```
-- Staff_Salaries

create or alter proc Staff_Salaries @Branch nvarchar(max)
as
begin
select s.name, s.salary, s.jobposition, b.location, s.EmploymentStatus
from staff s
join branch b
on s.BranchID = b.ID
where b.ID in (SELECT CAST(value AS INT)
              FROM STRING_SPLIT(@Branch, ','))
end
```


1. Trainer Salaries Report

REVIVE

Trainers Salaries			
Name	Salary	Seniority	Branch
Nabila ElSayed	10201	Senior	Menoufia
Khaled Mostafa	8158	Junior	Menoufia
Hany Shoukry	7432	Junior	Cairo
Maha ElGohary	10648	Senior	Giza
Mahmoud Elissa	7644	Junior	Menoufia
Amr Abdallah	8902	Junior	Menoufia
Kamal Elissa	10484	Senior	Alexandria
Zakaria Saleh	8547	Junior	Menoufia
Bassam Elsayed	10916	Senior	Menoufia
Dinaa ElNaggar	10895	Senior	Giza
Ayman ElMasry	7559	Junior	Alexandria
Gaber Hassan	8132	Junior	Menoufia
Ilyas Hassan	11899	Senior	Alexandria
Sawsan Mostafa	8266	Junior	Menoufia
Yassin ElGohary	10131	Senior	Giza
Nahed Ramy	9936	Senior	Menoufia
Yunus ElBaz	8559	Junior	Tanta
Ibrahim ElBaz	7383	Junior	Cairo
Hesham Abdelrahman	7211	Junior	Menoufia
Adel Khedr	11945	Senior	Cairo
Kawthar Tawfik	11194	Senior	Alexandria
Nada Barakat	8067	Junior	Alexandria
Anas Metwaly	10601	Senior	Menoufia
Reda Zahran	7342	Junior	Giza
Sahar Helmy	9815	Senior	Giza
Youssef Barakat	11530	Senior	Tanta
Mohsen Omran	7623	Junior	Menoufia
Ramadan Mohamed	11056	Senior	Tanta
Sayed ElBaz	8996	Junior	Giza
Laila Hassan	10322	Senior	Giza
Ayman ElNaggar	8983	Junior	Giza
Ayman Mostafa	11858	Senior	Menoufia

2. Staff Salaries Report

REVIVE

Staff Salaries				
Name	Salary	Job Position	Branch	Employment Status
Adel Emam	30000	General Manager	Cairo	Active
Sherine Ibrahim	22000	Facility Manager	Cairo	Active
Marwa Gaber	15000	HR Officer	Cairo	Active
Tamer Abdelaziz	14000	Accountant	Cairo	Active
Mohanad Hamed	11000	Massage Therapist	Cairo	Active
Yasser Fathy	11000	Massage Therapist	Cairo	Active
Ihab Morsi	11000	Massage Therapist	Cairo	Active
Sami Atiya	10000	Spa Attendant	Cairo	Active
Saber Gamal	10000	Spa Attendant	Cairo	Active
Yasser Ramadan	10000	Spa Attendant	Cairo	Active
Mohamed Anis	9000	Receptionist	Cairo	Active
Sara Fekry	9000	Receptionist	Cairo	Active
Maged Khedr	9000	Receptionist	Cairo	Active
Alaa Mounir	12000	Maintenance Technician	Cairo	Active
Wael ElGohary	12000	Maintenance Technician	Cairo	Active
Fatma Hassan	9500	Security Guard	Cairo	Active
Wesal Atef	9500	Security Guard	Cairo	Active
Sara Gamal	7000	Laundry Staff	Cairo	Active
Omar Selim	7000	Laundry Staff	Cairo	Active
Rasha Fahmy	7000	Laundry Staff	Cairo	Active
Mohanad Mahdy	7000	Laundry Staff	Cairo	Active
Ismail Abdelaziz	7000	Laundry Staff	Cairo	Active
Shawkat Kamel	7000	Laundry Staff	Cairo	Active
Bassem Osman	7000	Laundry Staff	Cairo	Active
Ezzat Zaki	7000	Laundry Staff	Cairo	Active
Sami Habib	7000	Laundry Staff	Cairo	Active
Fadi Atiya	6500	Cleaning Staff	Cairo	Active
Salah ElSayed	6500	Cleaning Staff	Cairo	Active
Noura Said	6500	Cleaning Staff	Cairo	Active
Omar Kamel	6500	Cleaning Staff	Cairo	Active
Magdy Hegazy	6500	Cleaning Staff	Cairo	Active
Omar Gamal	6500	Cleaning Staff	Cairo	Active
Fady ElMasry	6500	Cleaning Staff	Cairo	Active

SSRS Stored Procedures

3. Equipment Maintenance Report

Provides a detailed view of equipment maintenance activities, including maintenance type, cost, date, and equipment category.

This report helps branches track maintenance expenses and ensure equipment is properly serviced.

```
-- Equipment_Maintenace

create or alter proc Equipment_Maintenace @Branch nvarchar(max)
as
begin
select e.name, e.category, m.Type, em.cost, em.date, b.location
from branch b
join Equipment e
on b.ID = e.BranchID
join EquipmentMaintenance em
on e.ID = em.EquipmentID
join Maintenance m
on m.ID = em.MaintenanceID
where b.ID in (SELECT CAST(value AS INT)
              FROM STRING_SPLIT(@Branch, ','))
end
```

SSRS Stored Procedures

4. *Supplies Details Report*

Shows all supplies purchased by each branch, including quantity, price, supplier name, purchase date, and total spending.

Useful for monitoring inventory, purchasing patterns, and supplier relationships.

```
-- Supplies_Details

create or alter proc Supplies_Details @Branch nvarchar(max)
as
begin
select ss.name, ss.quantity, ss.price, sr.name as supplier, ss.PurchaseDate, b.location
from branch b
join Supplies ss
on b.ID = ss.BranchID
join Supplier sr
on ss.SupplierID = sr.ID
where b.ID in (SELECT CAST(value AS INT)
              FROM STRING_SPLIT(@Branch, ','))
end
```

3. Equipment Maintenance Report

R E V I V E					
Equipment Maintenance Details					
Equipment	Category	Type	Cost	Date	Branch
Leg Curl Machine (Lying)	Leg Machines	Preventive	832	2/23/2022	Cairo
Leg Curl Machine (Lying)	Leg Machines	Inspection	514	10/23/2023	Cairo
Glute Kickback Machine	Leg Machines	Preventive	851	9/23/2024	Cairo
Plate Loaded Squat Machine	Leg Machines	Preventive	808	6/23/2024	Cairo
Plate Loaded Squat Machine	Leg Machines	Inspection	590	2/14/2025	Cairo
Plate Loaded Squat Machine	Leg Machines	Corrective	2030	11/23/2023	Cairo
Incline Chest Press Machine	Chest / Upper Body Machines	Preventive	790	7/23/2022	Cairo
Incline Chest Press Machine	Chest / Upper Body Machines	Inspection	516	3/23/2024	Cairo
Assisted Pull-Up / Dip Machine	Back Machines	Preventive	794	9/23/2022	Cairo
Assisted Pull-Up / Dip Machine	Back Machines	Inspection	622	5/23/2024	Cairo
Selectorized Bicep Curl Machine	Shoulder / Arm Machines	Preventive	882	4/23/2022	Cairo
Selectorized Bicep Curl Machine	Shoulder / Arm Machines	Inspection	503	7/23/2024	Cairo
Power Rack	Functional / Multi-Station Equipment	Preventive	807	2/23/2022	Cairo
Power Rack	Functional / Multi-Station Equipment	Inspection	509	10/23/2023	Cairo
Seated Row Machine	Back Machines	Preventive	748	8/23/2023	Cairo
Seated Row Machine	Back Machines	Inspection	651	3/23/2025	Cairo
Seated Row Machine	Back Machines	Corrective	1516	10/23/2022	Cairo
Chest Fly / Rear Delt Combo Machine	Chest / Upper Body Machines	Preventive	806	5/23/2022	Cairo

4. Supplies Details Report

R E V I V E					
Supplies Details					
Supply	Quantity	Price	Supplier	Purchase Date	Branch
Supplements	31	80076.66	Scott Craig and Welch	5/27/2021	Cairo
Fitness Equipment	5	112070.33	Scott Craig and Welch	8/24/2021	Cairo
Recovery Equipment	8	116745.01	Rodriguez-Rivera	9/28/2021	Cairo
Weight Lifting Machines	2	564656.09	Rodriguez-Rivera	5/21/2022	Cairo
Recovery Equipment	7	93766.23	Rodriguez-Rivera	11/4/2022	Cairo
Dumbbell Set	8	243745.85	Rodriguez-Rivera	10/28/2025	Cairo
Safety & Protection Gear	15	77363.96	Costa PLC	11/20/2022	Cairo
Cardio Machines	3	1023199.37	Costa PLC	7/3/2024	Cairo
Fitness Equipment	6	213064.38	Costa PLC	9/4/2025	Cairo
Maintenance Tools	13	157203.65	Boyle Hughes and Montgomery	1/8/2022	Cairo
Recovery Equipment	5	34891.11	Boyle Hughes and Montgomery	3/23/2023	Cairo
Safety & Protection Gear	11	104909.15	Boyle Hughes and Montgomery	8/31/2023	Cairo
Safety & Protection Gear	14	40167.97	Meyer Inc	11/27/2022	Cairo
Supplements	57	310364.09	Meyer Inc	1/3/2023	Cairo
Hygiene Supplies	43	123520.04	Meyer Inc	11/27/2024	Cairo
Hygiene Supplies	12	16666.93	Johnson Avila and Robertson	9/29/2023	Cairo
Smart Fitness Devices	6	84559.79	Johnson Avila and Robertson	10/1/2025	Cairo
Fitness Equipment	8	120625.26	Brewer Curtis and Meyer	11/28/2021	Cairo
Maintenance Tools	20	70277.86	Brewer Curtis and Meyer	2/7/2025	Cairo
Supplements	34	97781.31	West PLC	7/29/2021	Cairo
Fitness Equipment	11	281242.82	West PLC	3/3/2023	Cairo
Gym Goods	24	47569.22	Fletcher PLC	7/11/2023	Cairo
Hygiene Supplies	24	43800.55	Fletcher PLC	7/13/2024	Cairo

SSRS Stored Procedures

5. Financial Performance Report

Summarizes yearly revenue for each branch, including:

- Total amount collected
- Year-over-year (YoY) change
- YoY percentage growth

This report helps assess financial health and track revenue trends across branches.

```
-- Financials

create or alter proc Financials @Branch nvarchar(max)
as
begin
SELECT
    b.location,
    YEAR([date]) AS [year],
    sum(TotalAmount) as 'Total Amount',
    sum(TotalAmount) - LAG(sum(TotalAmount)) OVER (PARTITION BY b.location ORDER BY YEAR([date])) AS yoy_change,
    (sum(TotalAmount) - LAG(sum(TotalAmount)) OVER (PARTITION BY b.location ORDER BY YEAR([date])))
    / NULLIF(LAG(sum(TotalAmount)) OVER (PARTITION BY b.location ORDER BY YEAR([date])), 0) AS yoy_percent
FROM PaymentDetails p
join branch b
on b.ID = p.BranchID
where b.ID in (SELECT CAST(value AS INT)
              FROM STRING_SPLIT(@Branch, ','))
group by b.location, YEAR([date])
end
```

SSRS Stored Procedures

6. Branch Feedback Report

Displays customer feedback categories and the average rating received per branch.

It helps management evaluate customer satisfaction and identify areas needing improvement.

```
-- Branch_Feedback

create or alter proc Branch_Feedback @Branch nvarchar(max)
as
begin
select b.Location, f.description, avg(tf.Rating) as 'Average Rating'
from branch b
join Trainee t
on b.ID = t.BranchID
join TraineeFeedback tf
on t.ID = tf.TraineeID
join Feedback f
on tf.FeedbackID = f.ID
--where b.ID in (SELECT CAST(value AS INT)
--      FROM STRING_SPLIT(@Branch, ','))
group by b.Location, f.Description
order by b.Location, [Average Rating] desc
end
```

5. Financial Performance Report



YOY Financial Analysis

Branch	Year	Total Amount	YOY Change	YOY Percent
Alexandria	2021	989,000.00		
Alexandria	2022	5,421,000.00	4432000	448.13%
Alexandria	2023	6,607,000.00	1186000	21.88%
Alexandria	2024	3,165,000.00	-3442000	-52.10%
Alexandria	2025	425,000.00	-2740000	-86.57%
Cairo	2021	1,422,000.00		
Cairo	2022	6,866,000.00	5444000	382.84%
Cairo	2023	8,297,000.00	1431000	20.84%
Cairo	2024	3,139,000.00	-5158000	-62.17%
Cairo	2025	694,000.00	-2445000	-77.89%
Giza	2021	1,209,000.00		
Giza	2022	6,718,000.00	5509000	455.67%
Giza	2023	7,756,000.00	1038000	15.45%
Giza	2024	3,615,000.00	-4141000	-53.39%
Giza	2025	518,000.00	-3097000	-85.67%
Menoufia	2021	442,000.00		
Menoufia	2022	2,332,000.00	1890000	427.60%
Menoufia	2023	2,729,000.00	397000	17.02%
Menoufia	2024	1,343,000.00	-1386000	-50.79%
Menoufia	2025	353,000.00	-990000	-73.72%
Tanta	2021	876,000.00		
Tanta	2022	2,873,000.00	1997000	227.97%
Tanta	2023	3,756,000.00	883000	30.73%
Tanta	2024	1,743,000.00	-2013000	-53.59%
Tanta	2025	292,000.00	-1451000	-83.25%

6. Branch Feedback Report



Branch Feedback

Branch	Feedback	AVG Rating
Alexandria	Operating Hours	3
Alexandria	Cleanliness	3
Alexandria	Crowd Management	3
Alexandria	Staff Friendliness	3
Alexandria	Equipment Maintenance	2
Alexandria	Trainer Performance	2
Alexandria	Class Variety	2
Alexandria	Customer Service	2
Alexandria	Pricing	2
Alexandria	Locker Room	2
Alexandria	Atmosphere	2
Alexandria	Equipment Availability	2
Alexandria	Hygiene Standards	2
Alexandria	Personal Training Quality	2
Alexandria	Music Choice	2
Cairo	Class Variety	3
Cairo	Locker Room	3
Cairo	Customer Service	3
Cairo	Cleanliness	3
Cairo	Equipment Maintenance	3
Cairo	Pricing	2
Cairo	Atmosphere	2
Cairo	Equipment Availability	2
Cairo	Music Choice	2
Cairo	Hygiene Standards	2
Cairo	Staff Friendliness	2
Cairo	Personal Training Quality	2



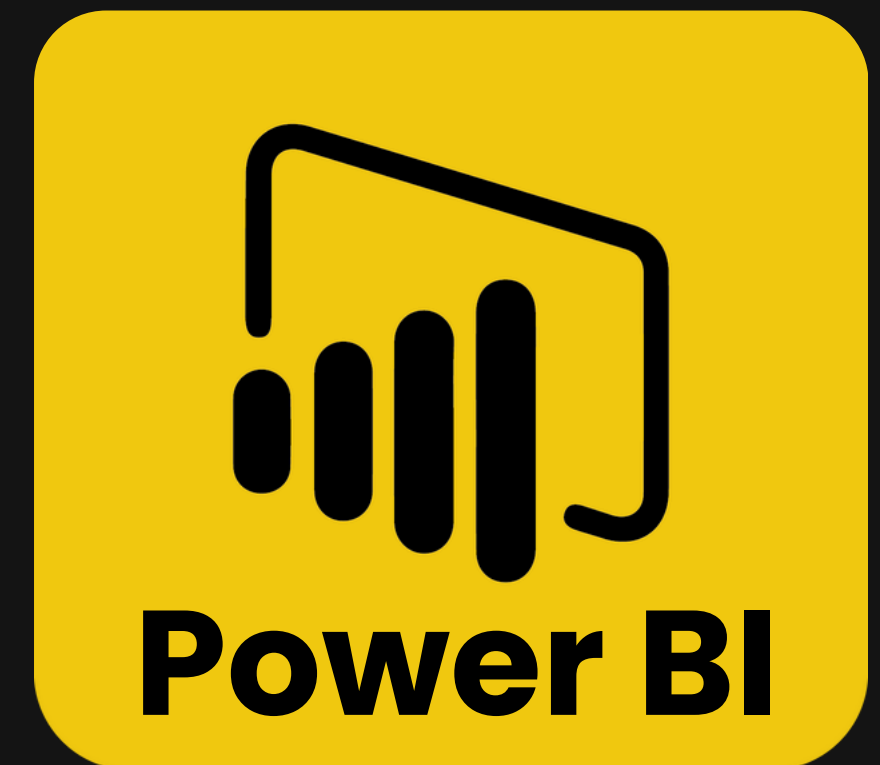
POWER BI DASHBOARDS

We built over 20 interactive dashboards using Power BI, incorporating a fully structured data model with fact and dimension tables.

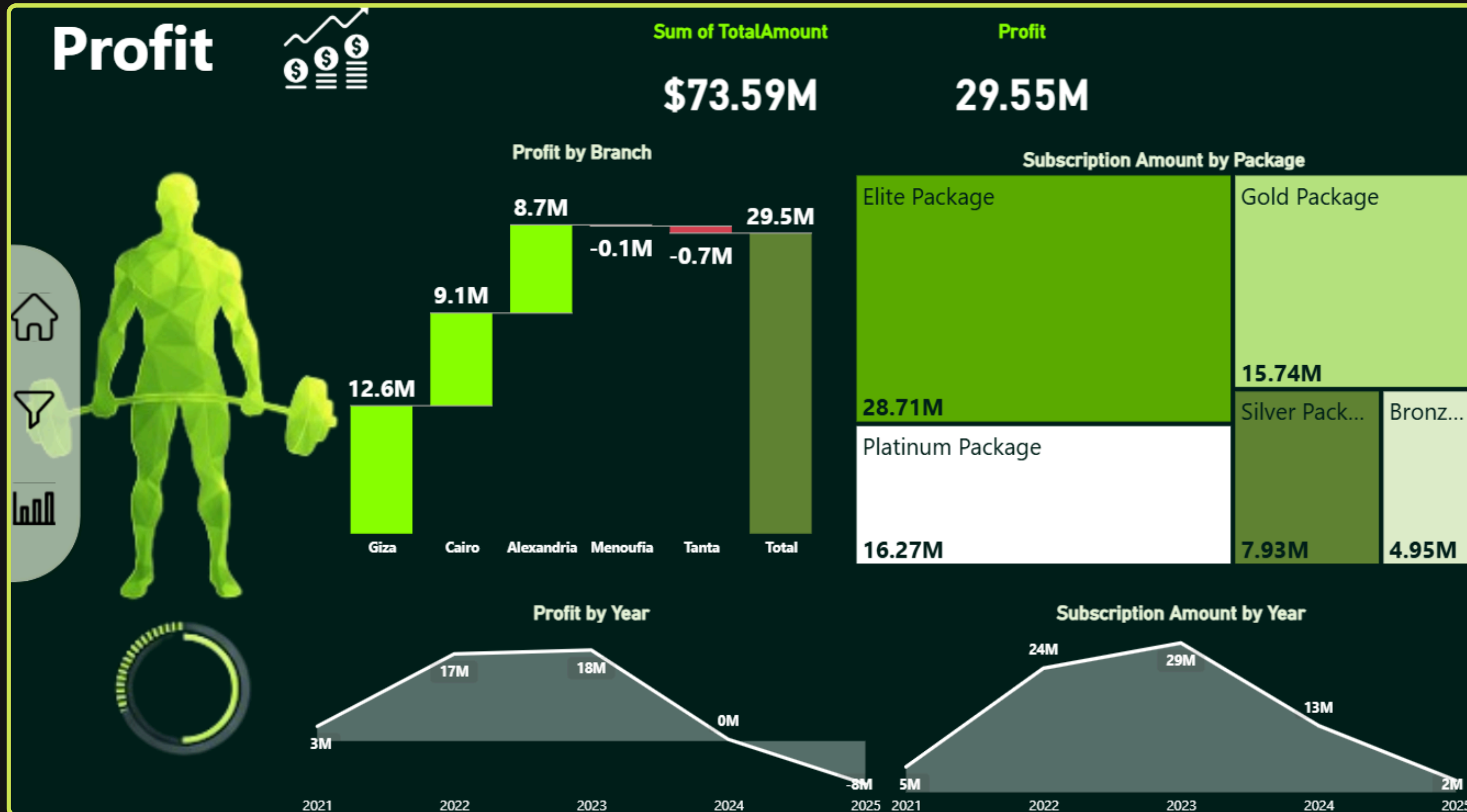
We applied several DAX measures to calculate KPIs, automate calculations, and enhance insights.

The dashboards were designed with a clean and user-friendly layout, using custom visuals, filters, and slicers to improve data exploration.

Overall, the Power BI solution provides clear, accurate, and dynamic insights to support decision-making across all branches.



Dashboards



Profit

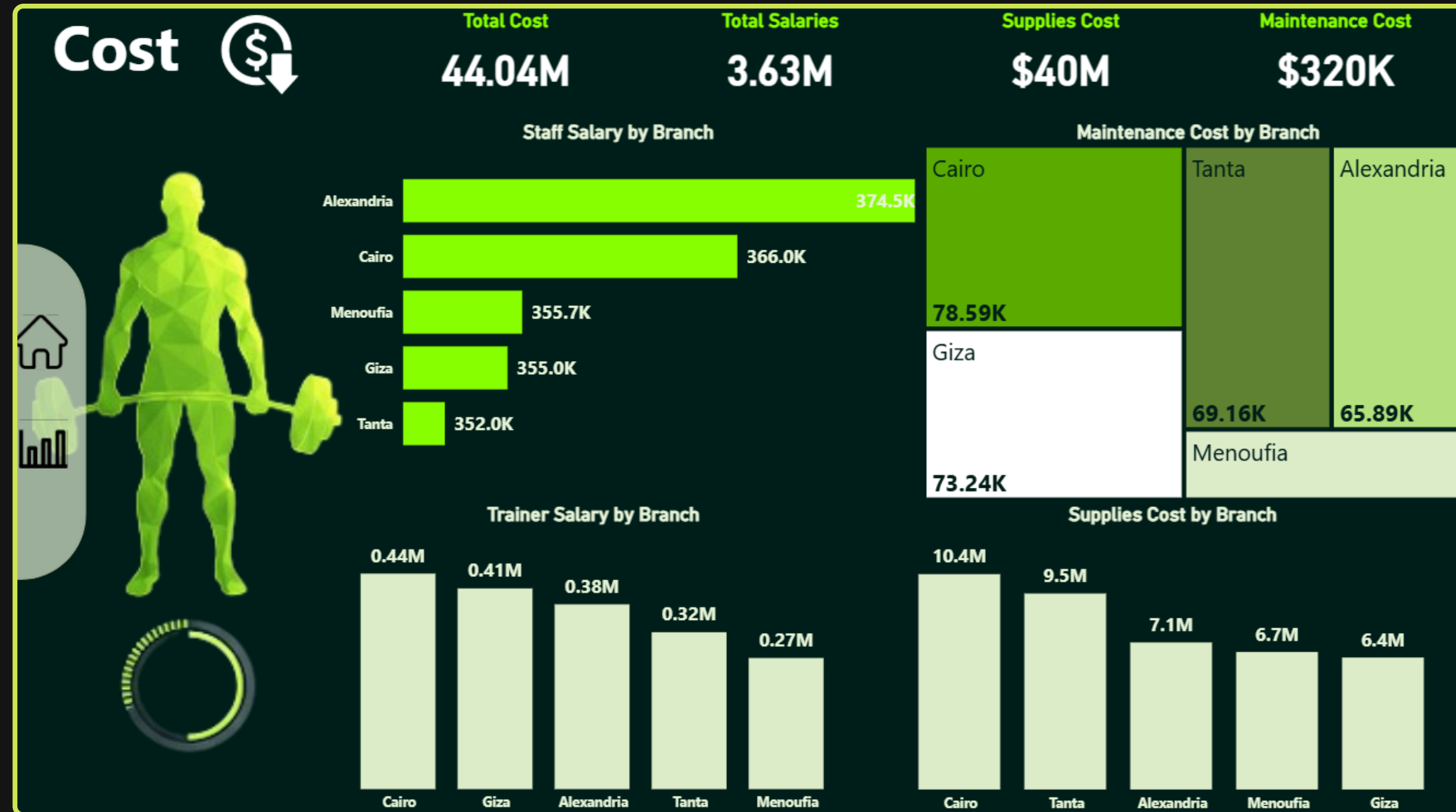
Top performance branch:
Giza: \$28.71 M

Loss Center:
Tanta: - 0.7 M
Menofia: - 0.1 M

Total Profit: 29.55 M

2024 was a profitless year
and **2025** ended with -8 M

Dashboards



Cost

Top Cost Driver Branch
Cairo is the highest cost driver across most categories leading in:

Trainer Salaries (0.44M)

Supplies Cost (10.4M)

Maintenance Cost (78.59K)

Highest Staff Salary Branch
Alexandria has the highest staff salary cost: **374.5k**

Maintenance Costs Are Very Low Relative to Total Cost
Maintenance = 320K

Dashboards

Subscription

Cairo & Giza Are Leading
Subscription Hubs

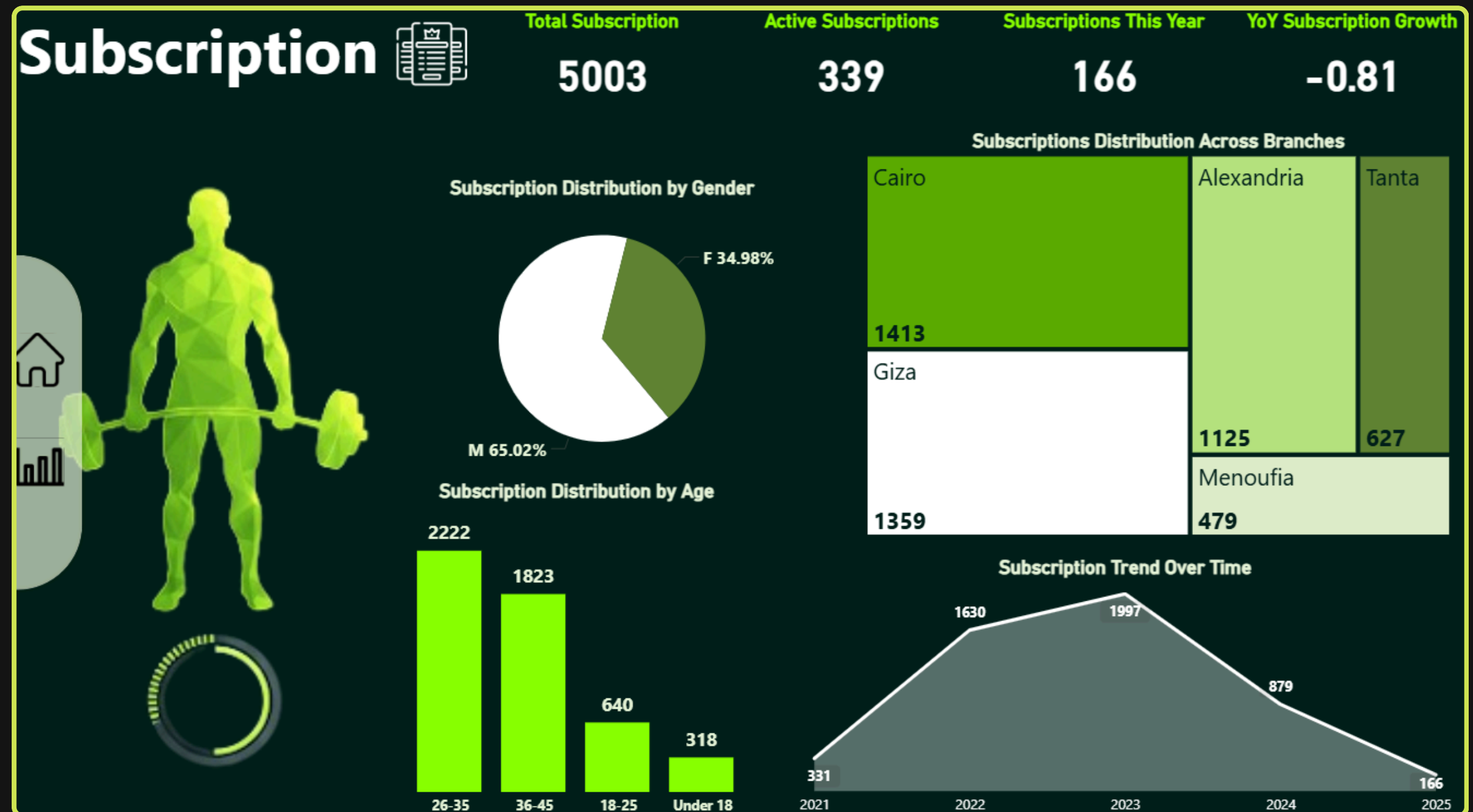
Cairo and Giza together contribute
over **55%** of total subscriptions
(1413 + 1359 = 2772)

2023 Was the Peak Subscription
Year

2023 achieved the highest
subscription volume: **1997**.

But then:

- **2024** dropped to **879**
- **2025** dropped to **166**



Dashboards

STAFF

Workforce is Highly Active (**94% Active Staff**)

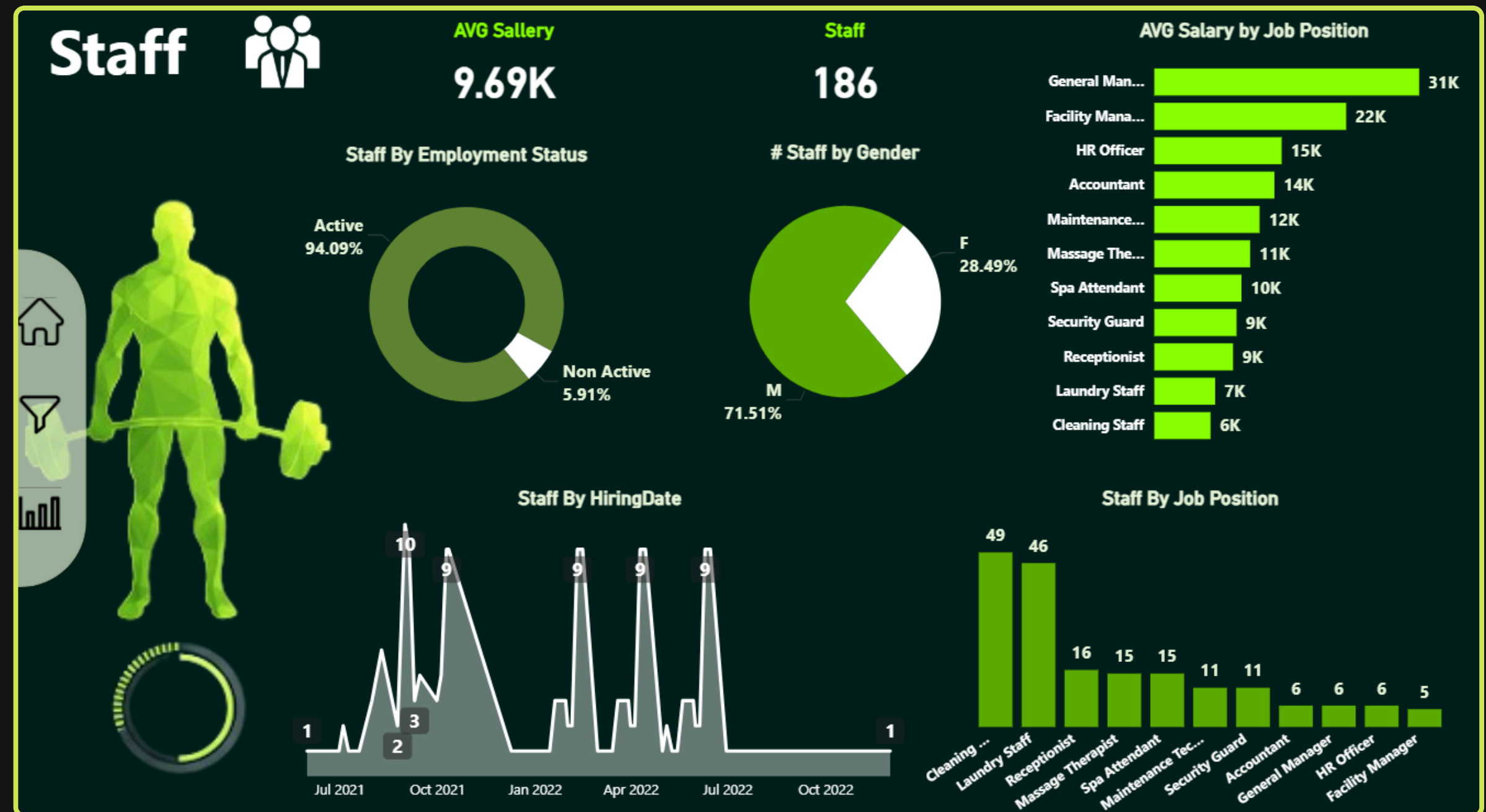
The organization maintains a very strong operational workforce.

Workforce is Male-Dominated
Males = **71.5%** Females = **28.5%**

General Manager Leads Salary Structure (**31K**)

Gym Trainers Are the Largest Staff Group

Gym Trainers = **49 employees**
Customer Service = **4**



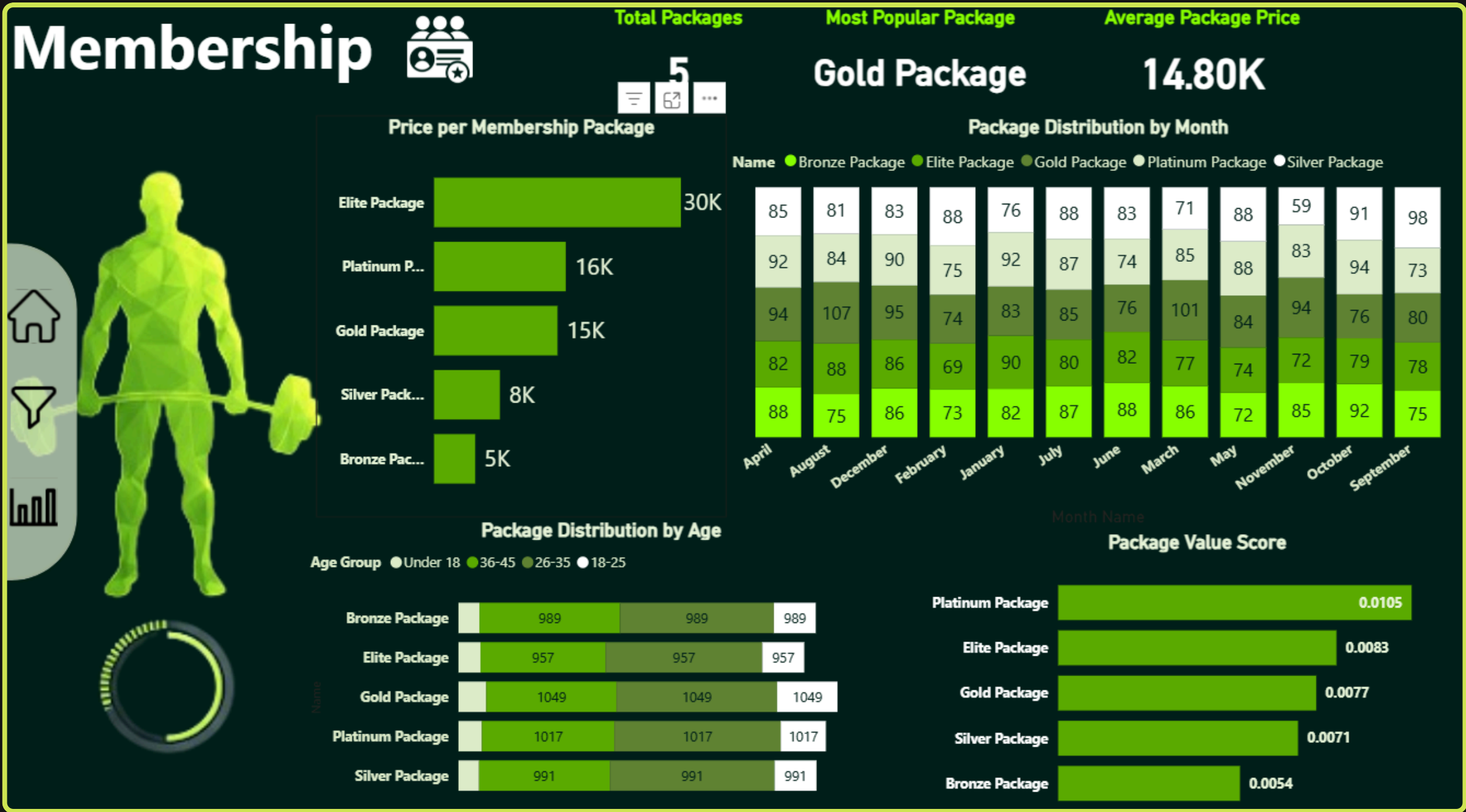
Dashboards

Membership

Gold Package Is the Top Seller
Despite Not Being the Highest
Value

Age Group 26-40 Is the Core
Customer Segment
Across all packages, age 26-40
dominates with counts between
597-1077 per package

Demand Peaks in April, June-July,
and October These months
consistently show higher package
signups

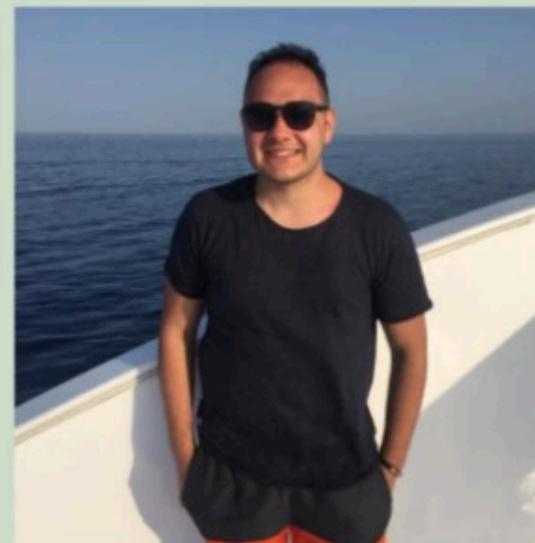




API

- Integrated Facebook Graph API to display profile image dynamically.
- Profile image updates automatically in the Power BI dashboard.
- Ensures real-time synchronization with the Facebook account.

Owners



Ahmed Khaled El-Doushy



Mohamed Ghazal





WEBSITE USING FLASK

We have developed a web application for managing a fitness center, including **trainee registration, attendance tracking, and payment management.**

Used **Flask** to manage the backend, and used **HTML, CSS, Bootstrap** for managing the frontend,

The application relies on our database stored on SQL Server. It consists of three pages:

- **Home** page with navigation buttons
- **Trainee** registration page
- **Attendance** registration page



HOME PAGE

It Serves as the **central navigation point**.

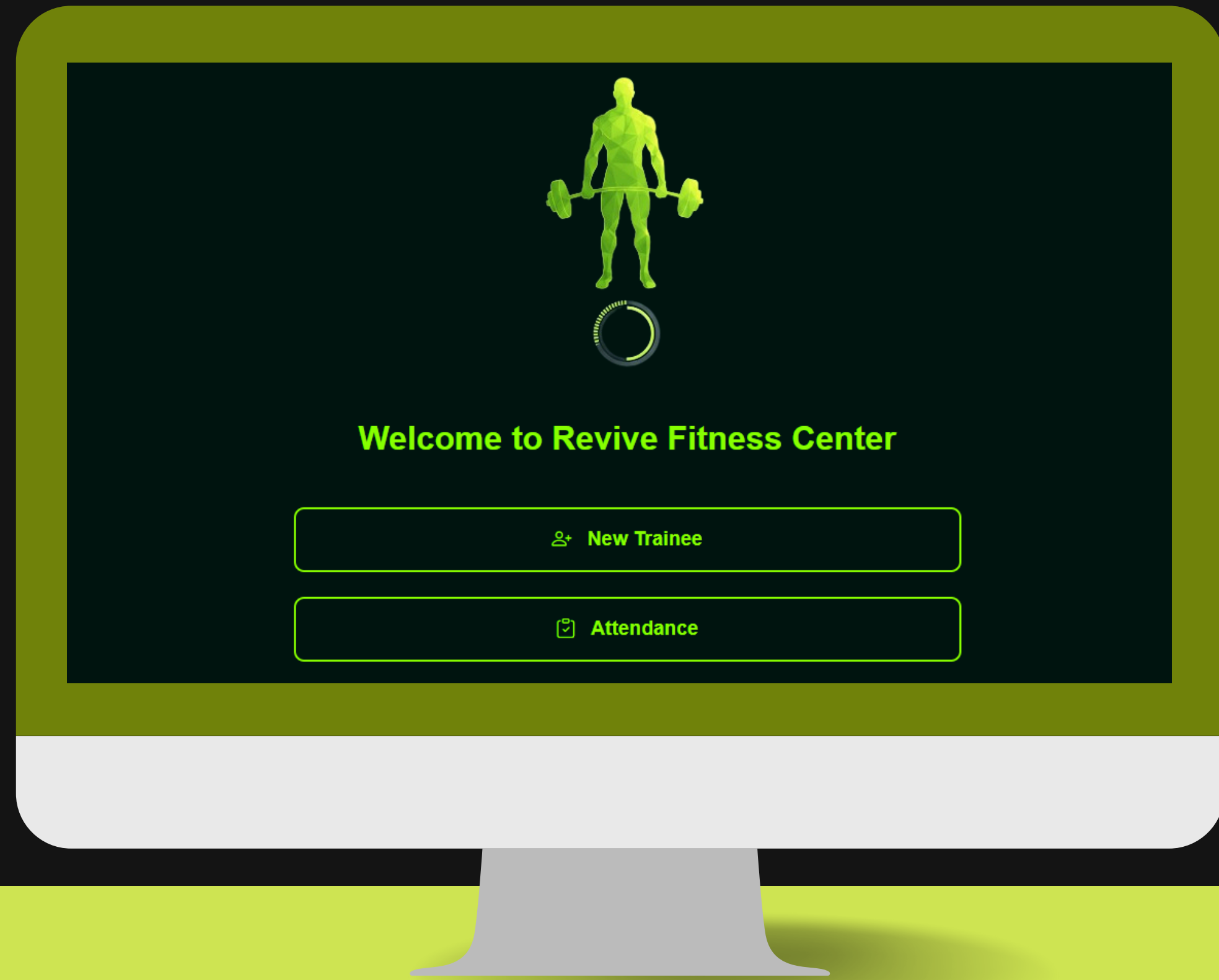
Designed a visually appealing interface with the logo prominently displayed.

Added two main buttons:

New Trainee: Opens trainee registration form.

Attendance: Opens attendance registration page.

Buttons include icons for better user experience.
Layout is responsive, works on different screen sizes.



TRAINEE REGISTRATION PAGE

it Registers a new trainee with all necessary information.

Trainee info: ID, name, gender, birth date, phone, email, branch.

Automatic Trainee ID: Fetches the latest ID + 1 from the database.

Membership subscription: Membership ID, status, start and expiry date.

Payment details: Payment date, invoice number (formatted as INV 6000), payment method, total amount.



The monitor displays a 'Trainee Registration Form' with the following sections:

- Trainee Info**
 - Trainee ID:
 - Full Name:
 - Gender:
 - Birth Date:
 - Phone Number:
 - Email Address:
 - Branch ID:
- Subscription Info**
 - Membership ID:
 - Status:
 - Start Date:
 - Expiry Date:
- Payment Details**
 - Payment Date:
 - Invoice Number:
 - Payment Method:
 - Total Amount:

A **Register Trainee** button is located at the bottom right of the form.



ATTENDANCE REGISTRATION PAGE

it Record daily attendance for trainees.

Select Trainee by **name & ID** from a drop down list.

Select **Branch** by location.

Automatically shows **current date & time**.

A stylized illustration of a computer monitor with a dark blue frame and a white base. The screen displays a form titled "Trainee Attendance Registration" in white text. The form has a dark blue background and contains three dropdown menus for "Trainee", "Branch", and "Date & Time". The "Date & Time" field is pre-filled with "12/11/2025 05:13 PM". Below the dropdowns is a white button with the text "Register Attendance".

Trainee Attendance Registration

Trainee
Select Trainee

Branch
Select Branch

Date & Time
12/11/2025 05:13 PM

Register Attendance





THANK YOU

